

function.

- This view leads developers to focus on issues of control and the decomposition of larger algorithms into smaller ones.
- As requirements change and the system grows, systems built with an algorithmic focus turn out to be very hard to maintain.

Object-Oriented Perspective

- The contemporary view of software development takes an object-oriented perspective.
- In this approach, the main building block of all software systems is the object or class.
- A class is a description of a set of common objects.
- Every object has identity, state, and behavior.
- Object-oriented development provides the conceptual foundation for assembling systems out of components using technology such as Java Beans or COM+.

An Overview of UML

- The Unified Modeling Language is a standard language for writing software blueprints. The UML may be used to visualize, specify, construct, and document the artifacts of a software-intensive system.
- The UML is appropriate for modeling systems ranging from enterprise information systems to distributed Web-based applications and even to hard real time embedded systems. It is a very expressive language, addressing all the views needed to develop and then deploy such systems.

The UML is a language for

- Visualizing
 - Specifying
 - Constructing
 - Documenting
- **Visualizing** The UML is more than just a bunch of graphical symbols. Rather, behind each symbol in the UML notation is a well-defined semantics. In this manner, one developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously
- **Specifying** means building models that are precise, unambiguous, and complete.
- **Constructing** the UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- **Documenting** a healthy software organization produces all sorts of artifacts in addition to raw executable code. These artifacts include
 - Requirements
 - Architecture
 - Design
 - Source code
 - Project plans
 - Tests
 - Prototypes
 - Releases

OBJECT ORIENTED ANALYSIS AND DESIGN

UNIT – I

Introduction to UML: Importance of modeling, principles of modeling, object oriented modeling, Conceptual model of the UML, Architecture, and Software Development Life Cycle.

UNIT – I

The UML is a graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system. The UML gives you a standard way to write a system's blueprints, covering conceptual things, such as business processes and system functions, as well as concrete things, such as classes written in a specific programming language, database schemas, and reusable software components.

Importance of modeling

A model is a simplification of reality. A model provides the blueprints of a system. A model may be structural, emphasizing the organization of the system, or it may be behavioral, emphasizing the dynamics of the system.

Aims of modeling

We build models so that we can better understand the system we are developing.

Through modeling, we achieve **four** aims.

1. Models help us to visualize a system as it is or as we want it to be.
2. Models permit us to specify the structure or behavior of a system.
3. Models give us a template that guides us in constructing a system.
4. Models document the decisions we have made.

We build models of complex systems because we cannot comprehend such a system in its entirety.

Principles of Modeling

There are **four** basic principles of model

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
2. Every model may be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is sufficient. Every nontrivial system is best approached through a small set of nearly independent models.

Object Oriented Modeling

In software, there are several ways to approach a model. The **two** most common ways are

1. Algorithmic perspective
2. Object-oriented perspective

Algorithmic Perspective

- The traditional view of software development takes an algorithmic perspective.
- In this approach, the main building block of all software is the procedure or

Conceptual model of UML

To understand the UML, you need to form a **conceptual model of the language**, and this requires learning three major elements:

- **Basic building blocks of the UML**
- **Rules**
- **Common Mechanisms in the UML**

Basic building blocks of the UML: Vocabulary of the UML can be defined

1. Things
2. Relationships
3. Diagrams

Things in the UML

There are **four** kinds of things in the UML:

- Structural things
- Behavioral things
- Grouping things
- Annotational things

Structural things are the nouns of UML models. These are the mostly static parts of a model, representing elements that are either conceptual or physical. In all, there are **seven** kinds of structural things.

1. Classes
2. Interfaces
3. Collaborations
4. Use cases
5. Active classes
6. Components
7. Nodes

Class

- It is a description of a set of objects that share the same attributes, operations, relationships, and semantics.
- A class implements one or more interfaces.
- Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations.

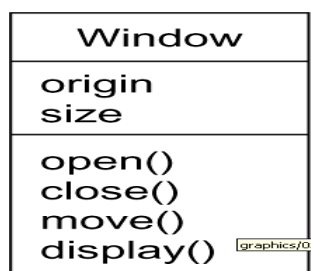


Fig: Class